

08 - Test Driven Development

1/10

Desarrollo Indicional

TDD

Se programa y al último se hacen las pruebas unitarias

Se escriben las pruebas unitarias antes de escribir el código

TDD: desarrollo guiado por pruebas de sw. Técnica que incluye: diseñar las pruebas unitarias, esas pruebas van a fallar → hago refactoring del código.

↳ RED - GREEN - REFACTOR

Beneficios

- Código robusto: código más predecible y limpio, más tolerable al cambio, más seguro, reduce la duplicación de código.
- código más barato de mantener
- acelera la velocidad del desarrollo y la posibilidad de despliegue continuo
- asegura cobertura de prueba → código de calidad.
- obliga a pensar en cómo usar el componente.
- promueve el desarrollo de componentes con alta cohesión y bajo acoplamiento.

Las 3 leyes de RDD

- ↳ no escribir una línea de código hasta que no hayas escrito antes un test case que falle
- ↳ no es necesario escribir más test de los necesarios para que falle. Un test que falle es ^{suficiente} ~~necesario~~.
- ↳ no escribir más código que el necesario para que el test pase.

Pasos por el TDD

- ↓ escribo un test
- ↓ hago que el test compile
- ↓ hago que el test falle
- ↓ hago un cambio por que el test pase
- ↓ corre el test por que pase
- ↓ refactoring

Patrones de TDD

→ Patrones de red bar

- one step test
- starter test
- explanation test
- learning test
- another test
- regression test

→ Patrones green bar

- fake it
- triangulation
- obvious implementation
- one to many

Refactoring: transformación del sw que preserva la estructura externa y mejora la estructura interna. Forma de introducir cambios reduciendo la posibilidad de introducir defectos.

- no cambia el comportamiento observable
- elimina la duplicación o complejidad innecesaria
- mejora la calidad de sw
- hace que el código sea más simple y fácil de entender
- flexibiliza el código, hace que sea más fácil de cambiar

Ej no 6 grupos impares: compars entrada

3/10

Pruebas de SW / Testing

PUD

- requerimientos → análisis → diseño → implementación → pruebas → despliegue

- el testing tiene entre 30% y 50% del costo de los proyectos
- la programación debería usar 33% - 35%

Error vs Defectos

Testing
un ~~se~~ exitoso encuentra defectos

Defecto: es un error que se trasladó de etapas. Ej, se generó en análisis y pasó a diseño. Los más costosos son los de requerimientos.

Error: se detecta y se corrige en la misma etapa donde se produjo.

Fallas: mal funcionamiento del sistema. A veces los errores o defectos, provocan fallas (y a veces no).

Testing metodológico vs Testing Ad Hoc

↓
Usa procesos

- planificación y diseño
- ejecución (reporte de defectos, herramientas como Jira)
- seguimiento y cierre

deberíamos
podemos adelantar esto!

↑ 50%

↓ 50%

Caso de Prueba

selec

escenario

resultado esperado

Niveles de prueba

- pruebas unitarias
- pruebas de integración / interfaz
- pruebas de sistemas o versión
- pruebas de aceptación de usuario (UAT)

Ambientes / Entornos

Desarrollo
Prueba / Staging
Pre producción
Producción

Comprar Entradas

Documento que justifique las decisiones!!

8/10

